BITS PILANI - WORK INTEGRATED LEARNING PROGRAMMES

Mobile Price Prediction

Applied Machine Learning | S1-26_DSEZG568

Group: GROUP_NO
Members and student IDs: [Add group details]

Open the Streamlit application
https://mobile-price-prediction-aml.streamlit.app/

We use mobile phone specifications to predict one of four price categories: **0 - Low, 1 - Medium, 2 - High, and 3 - Very high**. We compare Decision Tree, Random Forest, SVM and a voting ensemble, with Logistic Regression and a Dummy Classifier as baselines. The selected model is used in the Streamlit application.

1. Libraries and Dataset

We use NumPy and pandas for data handling, Matplotlib and seaborn for plots, and scikit-learn for preprocessing, training and evaluation. The two supplied CSVs are loaded below. The target column is `price_range`.

```
Target: price_range (four classes 0, 1, 2, 3). Original training records:
```

Columns 1-6 of 22

	id	battery_power	blue	clock_speed	dual_sim	fc
0	1	707	No	2.1	Yes	2
1	2	1218	No	1.8	Yes	1
2	3	1986	No	0.5	No	7
3	4	1290	No	0.5	No	0
4	5	1135	Yes	2.8	Yes	9

Columns 7-12 of 22

	four_g	int_memory	m_dep	mobile_wt	n_cores	pc
0	0	25	0.8	Med	3	17
1	1	14	0.5	Med	3	4
2	0	51	0.4	Low	3	9
3	0	3	0.1	Med	6	4
4	0	43	0.4	Med	1	11

Columns 13-18 of 22

	px_height	px_width	ram	sc_h	sc_w	talk_time
0	495	574	3838	9	2	7

	px_height	px_width	ram	sc_h	sc_w	talk_time
1	229	616	398	17	0	5
2	342	774	3165	5	0	13
3	600	1109	1214	9	8	15
4	690	1589	3204	18	13	6

Columns 19-22 of 22

	three_g	touch_screen	wifi	price_range
0	1	No	Yes	3
1	1	No	No	0
2	1	No	No	3
3	0	Yes	Yes	1
4	1	No	No	3

	package	executed_version
0	numpy	2.5.4
1	pandas	3.0.6
2	matplotlib	3.11.2
3	seaborn	0.13.2
4	scikit-learn	1.9.1
5	imbalanced-learn	0.14.2
6	joblib	1.6.0
7	streamlit	1.66.1
8	nbformat	5.11.1
9	nbclient	0.11.0
10	nbconvert	7.17.2
11	ipykernel	7.4.0
12	pytest	9.1.1

```
Python: 3.14.10 | Seed: 42
```

2. Data Understanding

The training data contains **2,000 rows and 22 columns**, including `id` and `price_range`. The test data contains **1,000 rows and 24 columns**. After removing the identifier and three empty-header test columns, both sets contain the same **20 predictors**. Four string columns contain Yes/No values; `mobile_wt` contains Low, Med and High weight categories.

Assumptions and evaluation setup

- `id` identifies a row and is excluded from the predictor matrix.
- The supplied test data is used for prediction because it has no target labels.
- We reserve 20% of the labeled data for final evaluation: **1,600 development rows and 400 holdout rows**, using stratification and `random_state=42`.

- Exploration and model selection use the development partition. The holdout is used after selecting the model with cross-validation.
- The target values are treated as four ordered price categories.

The following tables show the schema, two complete sample records and the split sizes.

	file	rows	columns	target_available
0	train	2000	22	True
1	unlabeled test	1000	24	False

	Predictor	Type	Distinct values	Missing
0	id	int64	2000	0
1	battery_power	int64	1094	0
2	blue	str	2	0
3	clock_speed	float64	26	0
4	dual_sim	str	2	0
5	fc	int64	20	0
6	four_g	int64	2	0
7	int_memory	int64	63	0
8	m_dep	float64	10	0
9	mobile_wt	str	3	0
10	n_cores	int64	8	0
11	pc	int64	21	0
12	px_height	int64	1137	0
13	px_width	int64	1109	0
14	ram	int64	1562	0
15	sc_h	int64	15	0
16	sc_w	int64	19	0
17	talk_time	int64	19	0
18	three_g	int64	2	0
19	touch_screen	str	2	0
20	wifi	str	2	0
21	price_range	int64	4	0

```
Two sample records:
```

Columns 1-6 of 22

	id	battery_power	blue	clock_speed	dual_sim	fc
0	1	707	No	2.100000	Yes	2
1	2	1218	No	1.800000	Yes	1

Columns 7-12 of 22

	four_g	int_memory	m_dep	mobile_wt	n_cores	pc
0	0	25	0.800000	Med	3	17
1	1	14	0.500000	Med	3	4

Columns 13-18 of 22

	px_height	px_width	ram	sc_h	sc_w	talk_time
0	495	574	3838	9	2	7
1	229	616	398	17	0	5

Columns 19-22 of 22

	three_g	touch_screen	wifi	price_range
0	1	No	Yes	3
1	1	No	No	0

	test_only_field	nulls	whitespace_only
0	Unnamed: 21	1000	0
1	Unnamed: 22	999	1
2	Unnamed: 23	1000	0

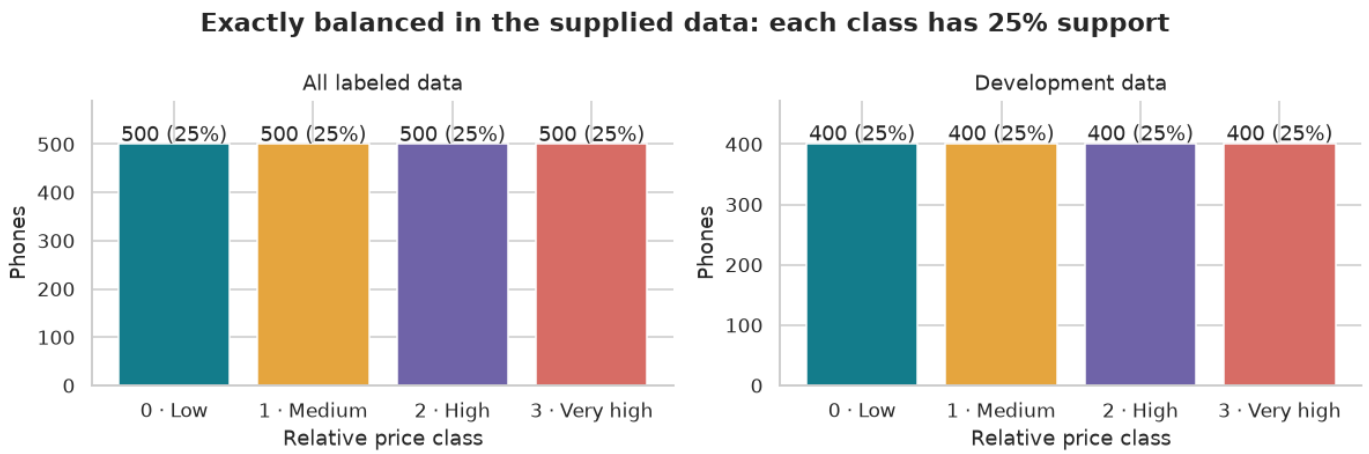
	object	shape
0	X_train (all labeled)	(2000, 20)
1	y_train	(2000,)
2	X_test / X_unlabeled_test	(1000, 20)
3	X_dev	(1600, 20)
4	y_dev	(1600,)
5	X_holdout	(400, 20)
6	y_holdout	(400,)

3. Exploratory Data Analysis

We examine class counts, summary statistics, individual feature distributions, relationships with the target and correlations. Related plots are grouped in one frame to make comparisons easier.

Units assumed: battery capacity in mAh, RAM in MB, storage in GB, clock speed in GHz, cameras in MP, resolution in pixels, screen dimensions and depth in cm, and talk time in hours. `mobile_wt` is used as a category.

Figure 1



Columns 1-6 of 9

	count	mean	std	min	25%	50%
battery_power	1600.0	1238.208	438.696	501.0	852.75	1228.5
clock_speed	1600.0	1.521	0.820	0.5	0.70	1.5
fc	1600.0	4.353	4.384	0.0	1.00	3.0
four_g	1600.0	0.522	0.500	0.0	0.00	1.0
int_memory	1600.0	31.858	18.271	2.0	16.00	31.0
m_dep	1600.0	0.503	0.289	0.1	0.20	0.5
n_cores	1600.0	4.555	2.288	1.0	3.00	5.0
pc	1600.0	9.942	6.096	0.0	5.00	10.0
px_height	1600.0	640.329	444.782	0.0	275.00	553.0
px_width	1600.0	1249.703	433.125	501.0	873.75	1243.5
ram	1600.0	2125.208	1091.241	256.0	1193.25	2130.5
sc_h	1600.0	12.238	4.159	5.0	9.00	12.0
sc_w	1600.0	5.746	4.275	0.0	2.00	5.0
talk_time	1600.0	10.976	5.462	2.0	6.00	11.0
three_g	1600.0	0.759	0.428	0.0	1.00	1.0

Columns 7-9 of 9

	75%	max	skew
battery_power	1608.50	1998.0	0.023
clock_speed	2.20	3.0	0.195
fc	7.00	19.0	1.013
four_g	1.00	1.0	-0.090
int_memory	48.00	64.0	0.079
m_dep	0.80	1.0	0.074
n_cores	7.00	8.0	-0.010
pc	15.00	20.0	0.016

	75%	max	skew
px_height	945.50	1960.0	0.680
px_width	1623.00	1997.0	0.020
ram	3081.25	3993.0	0.005
sc_h	16.00	19.0	-0.063
sc_w	9.00	18.0	0.616
talk_time	16.00	20.0	0.009
three_g	1.00	1.0	-1.211

	count	unique	top	freq
blue	1600	2	No	808
dual_sim	1600	2	Yes	810
touch_screen	1600	2	No	806
wifi	1600	2	Yes	804
mobile_wt	1600	3	Med	668

Each class has **500 records**, giving 400 development and 100 holdout records per class. The labels are exactly balanced. We still use macro-F1 for model selection because it gives equal importance to each class's precision and recall.

The distributions show a wide spread of RAM and battery capacity, discrete connectivity flags, and three weight bands. Front-camera values and pixel height are more skewed than RAM or battery capacity. Scaling changes the units of a feature; it does not remove skewness. Recorded zeros are examined in the next section.

Figure 2

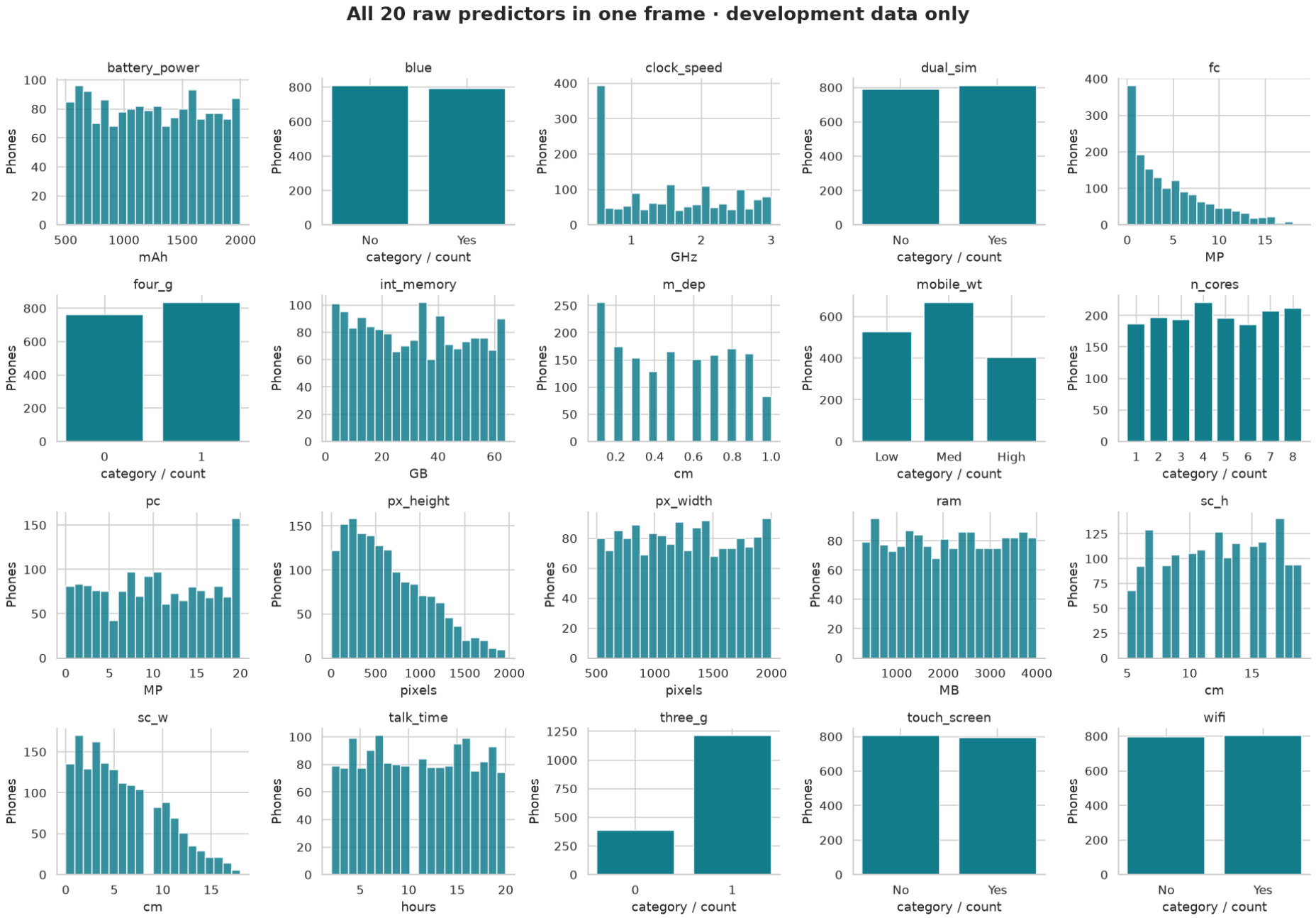
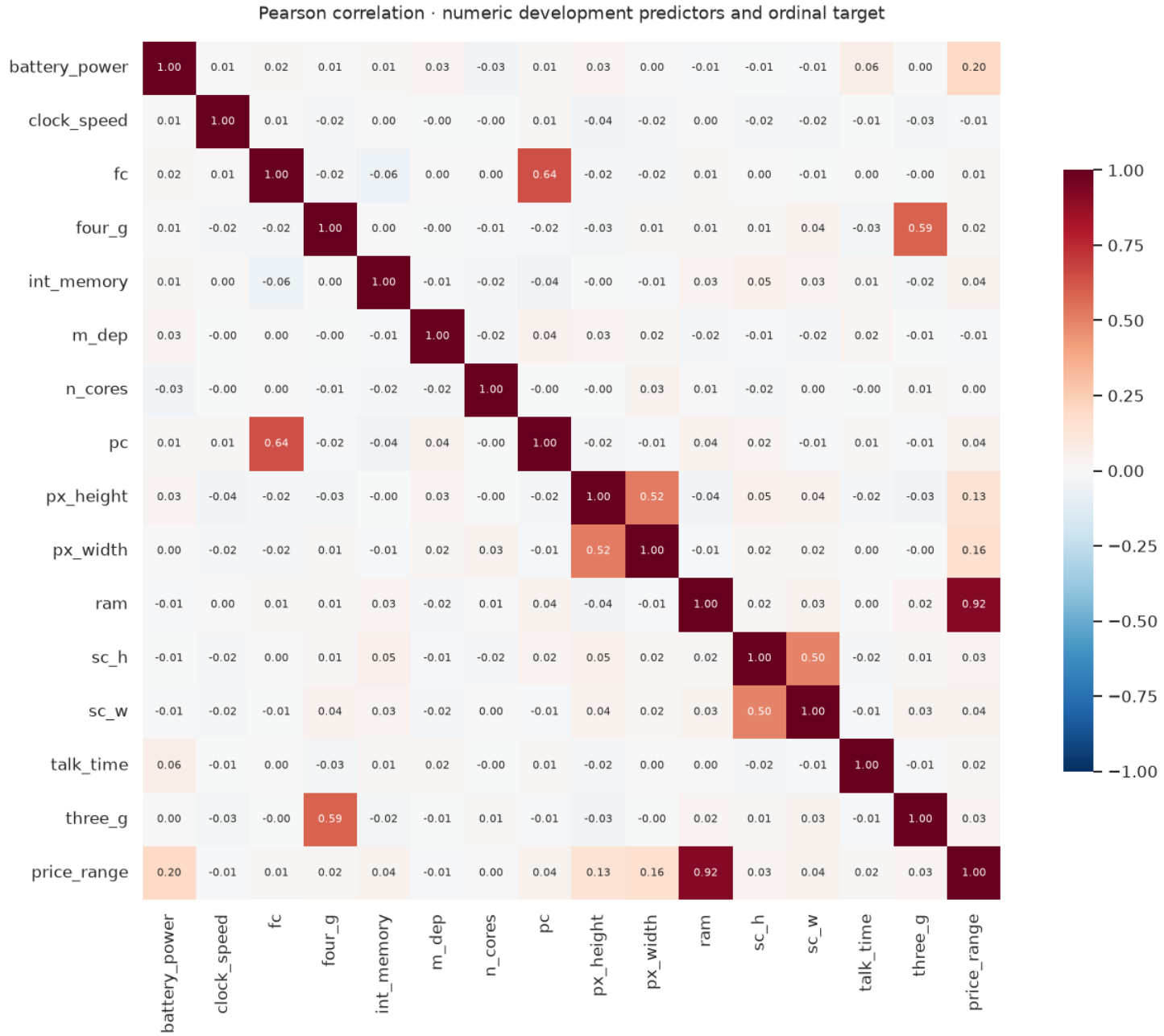

All 20 raw predictors in one frame · development data only
battery_power
Phones
mAh
blue
category / count
No
Yes
clock_speed
GHz
dual_sim
fc
MP
four_g
int_memory
GB
m_dep
cm
mobile_wt
Low
Med
High
n_cores
pc
MP
px_height
pixels
px_width
ram
MB
sc_h
sc_w
talk_time
hours
three_g
touch_screen
wifi

Figure 3

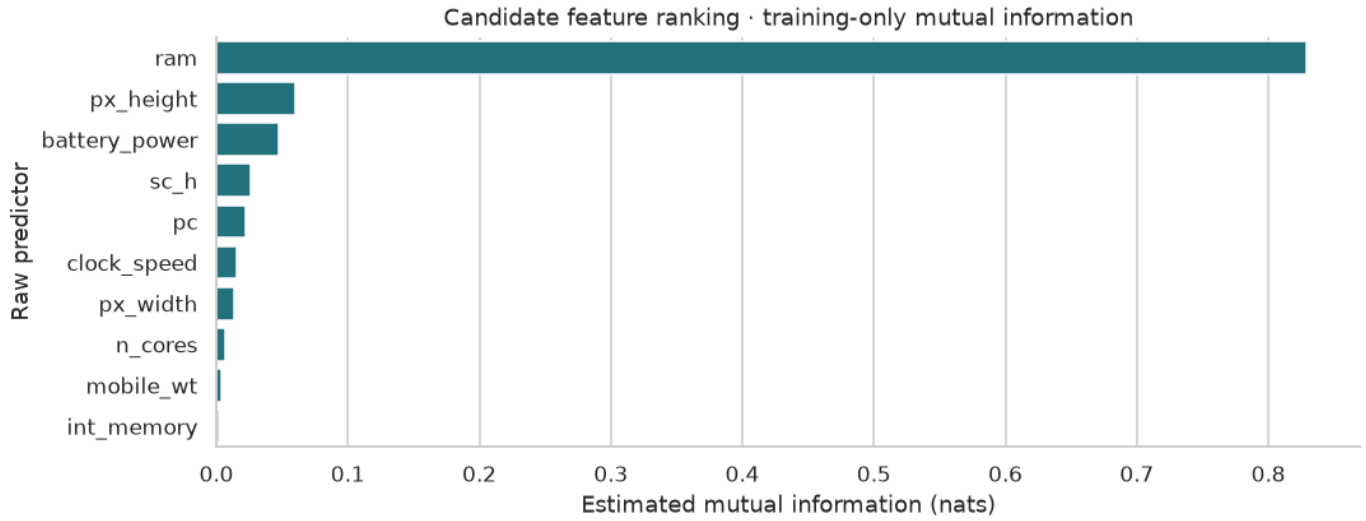

Price class vs memory, battery and pixel characteristics
ram across classes
ram (MB)
Price class
battery_power across classes
battery_power (mAh)
Price class
px_height across classes
px_height (pixels)
Price class
ram × battery_power
battery_power (mAh)
ram (MB)
price_range
0
1
2
3
ram × px_width
px_width (pixels)
ram (MB)
px_height × px_width
px_width (pixels)
px_height (pixels)

Figure 4


Pearson correlation · numeric development predictors and ordinal target
battery_power 1.00 0.01 0.02 0.01 0.01 0.03 -0.03 0.01 0.03 0.00 -0.01 -0.01 -0.01 0.06 0.00 0.20
clock_speed 0.01 1.00 0.01 -0.02 0.00 -0.00 -0.00 0.01 -0.04 -0.02 0.00 -0.02 -0.02 -0.01 -0.03 -0.01
fc 0.02 0.01 1.00 -0.02 -0.06 0.00 0.00 0.64 -0.02 -0.02 0.01 0.00 -0.01 0.00 -0.00 0.01
four_g 0.01 -0.02 -0.02 1.00 0.00 -0.00 -0.01 -0.02 -0.03 0.01 0.01 0.01 0.04 -0.03 0.59 0.02
int_memory 0.01 0.00 -0.06 0.00 1.00 -0.01 -0.02 -0.04 -0.00 -0.01 0.03 0.05 0.03 0.01 -0.02 0.04
m_dep 0.03 -0.00 0.00 -0.00 -0.01 1.00 -0.02 0.04 0.03 0.02 -0.02 -0.01 -0.02 0.02 -0.01 -0.01
n_cores -0.03 -0.00 0.00 -0.01 -0.02 -0.02 1.00 -0.00 -0.00 0.03 0.01 -0.02 0.00 -0.00 0.01 0.00
pc 0.01 0.01 0.64 -0.02 -0.04 0.04 -0.00 1.00 -0.02 -0.01 0.04 0.02 -0.01 0.01 -0.01 0.04
px_height 0.03 -0.04 -0.02 -0.03 -0.00 0.03 -0.00 -0.02 1.00 0.52 -0.04 0.05 0.04 -0.02 -0.03 0.13
px_width 0.00 -0.02 -0.02 0.01 -0.01 0.02 0.03 -0.01 0.52 1.00 -0.01 0.02 0.02 0.00 -0.00 0.16
ram -0.01 0.00 0.01 0.01 0.03 -0.02 0.01 0.04 -0.04 -0.01 1.00 0.02 0.03 0.00 0.02 0.92
sc_h -0.01 -0.02 0.00 0.01 0.05 -0.01 -0.02 0.02 0.05 0.02 0.02 1.00 0.50 -0.02 0.01 0.03
sc_w -0.01 -0.02 -0.01 0.04 0.03 -0.02 0.00 -0.01 0.04 0.02 0.03 0.50 1.00 -0.01 0.03 0.04
talk_time 0.06 -0.01 0.00 -0.03 0.01 0.02 -0.00 0.01 -0.02 0.00 0.00 -0.02 -0.01 1.00 -0.01 0.02
three_g 0.00 -0.03 -0.00 0.59 -0.02 -0.01 0.01 -0.01 -0.03 -0.00 0.02 0.01 0.03 -0.01 1.00 0.03
price_range 0.20 -0.01 0.01 0.02 0.04 -0.01 0.00 0.04 0.13 0.16 0.92 0.03 0.04 0.02 0.03 1.00
battery_power
clock_speed
fc
four_g
int_memory
m_dep
n_cores
pc
px_height
px_width
ram
sc_h
sc_w
talk_time
three_g
price_range
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

RAM has a development-set Pearson correlation of **0.916** with the ordered class labels. RAM distributions shift upward across the classes, while battery capacity and resolution provide additional separation. Overlap in the plots motivates using several features together. The model comparison tests whether nonlinear boundaries improve on Logistic Regression.

Figure 5



	Predictor	mutual_information
0	ram	0.8286
1	px_height	0.0601
2	battery_power	0.0470
3	sc_h	0.0264
4	pc	0.0226
5	clock_speed	0.0152
6	px_width	0.0133
7	n_cores	0.0071

	Finding	Evidence	Action
0	Class counts	500 each / 400 each in development	No aggressive resampling; compare controls in CV
1	RAM	Pearson r=0.916; top MI features: ram, px_height, battery_power	Retain, scale for SVM/LR
2	Battery / pixels	Class overlap and multivariate variation	Test jointly in classifiers; area ablation
3	Categoricals	Four Yes/No predictors and three weight bands	Binary encoding and one-hot weight bands
4	Nonlinearity	Visual evidence alone is inconclusive	Compare RBF/tree families with linear baseline
5	Zeros / extrema	Recorded finite values; semantics partly undocumented	Audit, retain, avoid unsafe ratios

Feature ranking: mutual information measures how much a feature helps distinguish the classes, including dependence that a linear correlation may miss. We use it as an exploratory ranking and retain the original predictors for model comparison. Later, we compare this ranking with Random Forest importance and permutation importance.

4. Data Preprocessing

We check missing values, duplicates, negative values, camera consistency, screen dimensions, skewness and IQR outlier flags before fitting the models.

Assumptions used

- Recorded zero values are retained as supplied. Camera zeros can indicate absence; zero screen width or pixel height is treated as a recorded value. We avoid ratios that would divide by zero.
- Extreme specifications are retained when they fall within the dataset's ranges. An IQR flag alone is not used to delete a record.
- Yes/No values use binary encoding: **No = 0, Yes = 1**. Weight categories use **one-hot encoding**, so the model does not assume equal numerical gaps between bands.
- Numeric median imputation and categorical mode imputation are included in the pipeline. They do not change the supplied complete predictors.

For Logistic Regression and SVM, `StandardScaler` subtracts the training mean and divides by the training standard deviation. This prevents large-unit features such as RAM from dominating regularization or distance calculations. Trees choose split thresholds and therefore use unscaled numeric inputs. The encoder, imputer and scaler are fitted separately inside each cross-validation training fold.

	check	count
0	True missing cells (all train)	0
1	Missing raw test predictors	0
2	Duplicate complete training rows	0
3	Duplicate training predictor rows	0
4	Duplicate development predictor rows	0
5	Duplicate train IDs	0
6	Negative numeric values (development)	0
7	Front camera exceeds primary (development)	0
8	4G enabled without 3G (development)	0
9	Screen width exceeds height (development)	0

	field	train_zero_count	development_zero_count	test_zero_count
0	sc_w	180	135	112
1	px_height	2	2	2
2	fc	474	381	210
3	pc	101	81	40

	skew	IQR_flag_count	action
battery_power	0.0234	0	Retain; no documented invalid value
clock_speed	0.1952	0	Retain; no documented invalid value
fc	1.0127	15	Retain; no documented invalid value
four_g	-0.0902	0	Retain; no documented invalid value

	skew	IQR_flag_count	action
int_memory	0.0793	0	Retain; no documented invalid value
m_dep	0.0740	0	Retain; no documented invalid value
n_cores	-0.0096	0	Retain; no documented invalid value
pc	0.0159	0	Retain; no documented invalid value
px_height	0.6796	1	Retain; no documented invalid value
px_width	0.0197	0	Retain; no documented invalid value
ram	0.0053	0	Retain; no documented invalid value
sc_h	-0.0630	0	Retain; no documented invalid value
sc_w	0.6163	0	Retain; no documented invalid value
talk_time	0.0090	0	Retain; no documented invalid value
three_g	-1.2107	386	Retain; no documented invalid value

	raw_binary	label_style_code
0	No	0
1	Yes	1

	weight_band	weight_Low	weight_Med	weight_High
0	Low	1	0	0
1	Med	0	1	0
2	High	0	0	1

	operation	fit_boundary
0	Drop Unnamed columns	Deterministic schema rule
1	Exclude id	Deterministic role rule
2	Impute median/mode	CV training fold
3	Encode categories	CV training fold
4	Standardize numeric	CV training fold (LR/SVM)
5	Area products	Deterministic; choice by development CV
6	Balance classes	CV training fold only

Class balancing. We compare no balancing, balanced class weights and random oversampling within the training folds. Each five-fold training partition has 320 examples per class. Therefore balanced weights are 1 and random oversampling adds no rows. Random oversampling duplicates existing observations, preserving categorical values. SMOTENC is an alternative for mixed numeric/categorical data when classes are imbalanced; synthetic sampling is unnecessary here.

Feature engineering. We test two products: `pixel_area = px_height * px_width` and `screen_area = sc_h * sc_w`. Both are safe when inputs contain zero. For each comparison we keep the model's tuned hyperparameters fixed and use the same folds. We retain the extra features only if mean CV macro-F1 improves by more than 0.002. This threshold favors the simpler feature set when the measured change is small.

5. Model Training and Comparison

Classifier	Reason for including it
Decision Tree	Learns feature thresholds and nonlinear decision rules
Random Forest	Combines trees to reduce the variation of a single tree
SVM with RBF kernel	Tests flexible, smooth class boundaries
Soft Voting	Combines Logistic Regression, Random Forest and SVM probabilities
Logistic Regression	Provides a linear classification baseline
Dummy Classifier	Provides a simple reference score by predicting the most frequent class

We tune the models with **five-fold stratified cross-validation** on the 1,600 development rows. Macro-F1 is the selection metric, with accuracy used to break an exact tie. Every candidate uses the same folds. The table shows mean scores and their standard deviation across folds.

SVM probabilities are obtained with sigmoid calibration using five inner folds. The complete preprocessing and SVM pipeline is fitted within those folds. The voting model averages its members' class probabilities. Tuning and feature choices reuse the development folds, so the separate holdout provides the final evaluation.

	Classifier	CV macro-F1	F1 std	CV accuracy	Accuracy std	Training F1	Area features
0	Logistic Regression	0.9612	0.0127	0.9612	0.0126	0.9880	False
1	Soft Voting	0.9443	0.0174	0.9444	0.0172	0.9928	False
2	SVM (RBF)	0.9156	0.0172	0.9156	0.0169	0.9800	False
3	Random Forest	0.8917	0.0092	0.8912	0.0096	1.0000	False
4	Decision Tree	0.8404	0.0071	0.8400	0.0078	0.9362	False
5	Dummy	0.1000	0.0000	0.2500	0.0000	0.1000	False

	Classifier	Candidates	Selected parameters
0	Dummy	1	{}
1	Logistic Regression	4	{"C": 100.0}
2	Decision Tree	6	{"max_depth": 8, "min_samples_leaf": 5}
3	Random Forest	8	{"max_depth": 12, "max_features": 0.7, "min_samples_leaf": 1}
4	SVM (RBF)	9	{"C": 10.0, "gamma": 0.01}
5	Soft Voting	2	{"weights": [1, 1, 1]}

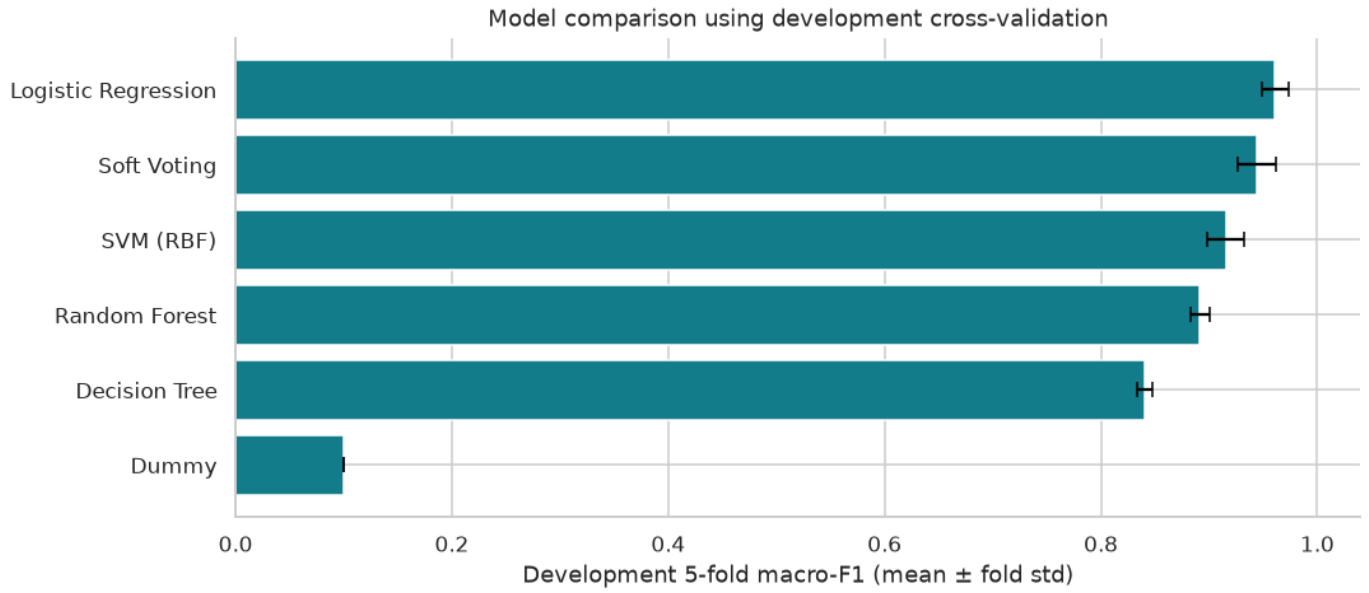
```
CV-selected model: Logistic Regression. Experiment runtime: 25.6 seconds.
```

	Classifier	Feature set	CV macro-F1	F1 std	CV accuracy	Change in F1	Retained
0	Logistic Regression	Original features	0.9612	0.0127	0.9612	0.0000	True
1	Logistic Regression	Original + pixel_area + screen_area	0.9618	0.0124	0.9619	0.0006	False
2	SVM (RBF)	Original features	0.9156	0.0172	0.9156	0.0000	True
3	SVM (RBF)	Original + pixel_area + screen_area	0.9111	0.0228	0.9113	-0.0044	False

	Method	CV macro-F1	F1 std	CV accuracy	Added rows/fold
0	No balancing	0.9612	0.0127	0.9612	0.0
1	Class weight = balanced	0.9612	0.0127	0.9612	0.0
2	RandomOverSampler (inside CV)	0.9612	0.0127	0.9612	0.0

Feature comparison: Logistic Regression: Original features; SVM (RBF): Original features. Balancing macro-F1 spread = **0.000000**; all oversampling fold additions = **0** rows. Each class has the same support, so the original class distribution is retained. The comparison confirms that balancing makes no change here.

Figure 6



	Encoded predictor	Tree importance
0	numeric__ram	0.6426
1	numeric__battery_power	0.1153
2	numeric__px_height	0.0748
3	numeric__px_width	0.0725
4	numeric__int_memory	0.0129
5	numeric__talk_time	0.0095
6	numeric__pc	0.0093

	Encoded predictor	Tree importance
7	numeric__sc_w	0.0090
8	numeric__sc_h	0.0090
9	numeric__clock_speed	0.0080

Selected model: Logistic Regression, with development CV macro-F1 **0.9612 ± 0.0127**. Impurity importance is training-only and can favor continuous/high-cardinality variables or split credit between correlated features. It complements the mutual-information ranking. We evaluate this configuration on the holdout in the next section.

6. Performance Evaluation

We evaluate the selected model on the **400 labeled holdout records** after fitting it on the development data.

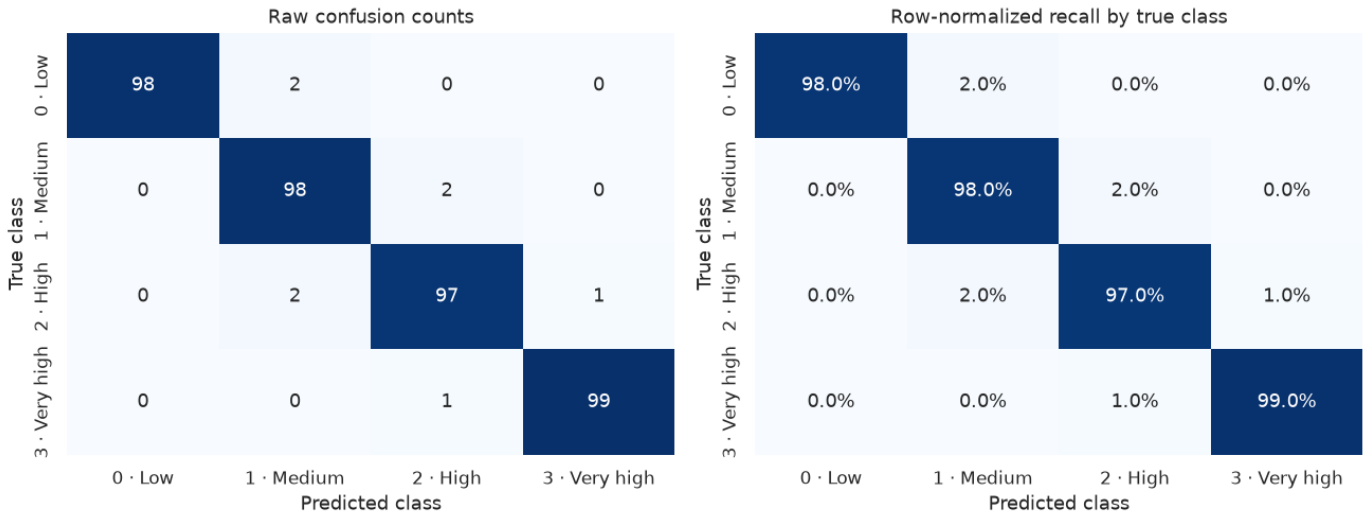
- **Accuracy:** fraction of all predictions that are correct.
- **Precision for a class:** correct predictions of that class divided by all predictions of it.
- **Recall for a class:** correct predictions of that class divided by its actual support.
- **F1:** harmonic mean of precision and recall.
- **Macro average:** equal average across classes. **Weighted average:** average weighted by class support.

The confusion matrix uses actual classes as rows and predicted classes as columns. Row normalization shows the fraction of each actual class assigned to every predicted class.

	accuracy	Macro precision	Macro recall	Macro-F1	Weighted precision	Weighted recall	Weighted F1
0	0.98	0.9802	0.98	0.98	0.9802	0.98	0.98

	precision	recall	f1-score	support
0 · Low	1.0000	0.98	0.9899	100.0
1 · Medium	0.9608	0.98	0.9703	100.0
2 · High	0.9700	0.97	0.9700	100.0
3 · Very high	0.9900	0.99	0.9900	100.0
macro avg	0.9802	0.98	0.9800	400.0
weighted avg	0.9802	0.98	0.9800	400.0

Figure 7



	evaluation	macro_F1
0	Development fit (resubstitution)	0.9850
1	Development CV mean	0.9612
2	Untouched holdout	0.9800

	true_class	predicted_class	count
0	0	1	2
1	1	2	2
2	2	1	2
3	2	3	1
4	3	2	1

Error discussion. 8/400 holdout phones are misclassified; 8 errors are between adjacent classes. The lowest class F1 is **2 · High: 0.9700**. Accuracy Wilson 95% interval is **[0.961, 0.990]**. The resubstitution-to-holdout macro-F1 gap is **0.0050**; compare this with the CV mean and variability above. The small gap suggests that the selected model fits the development and holdout partitions consistently.

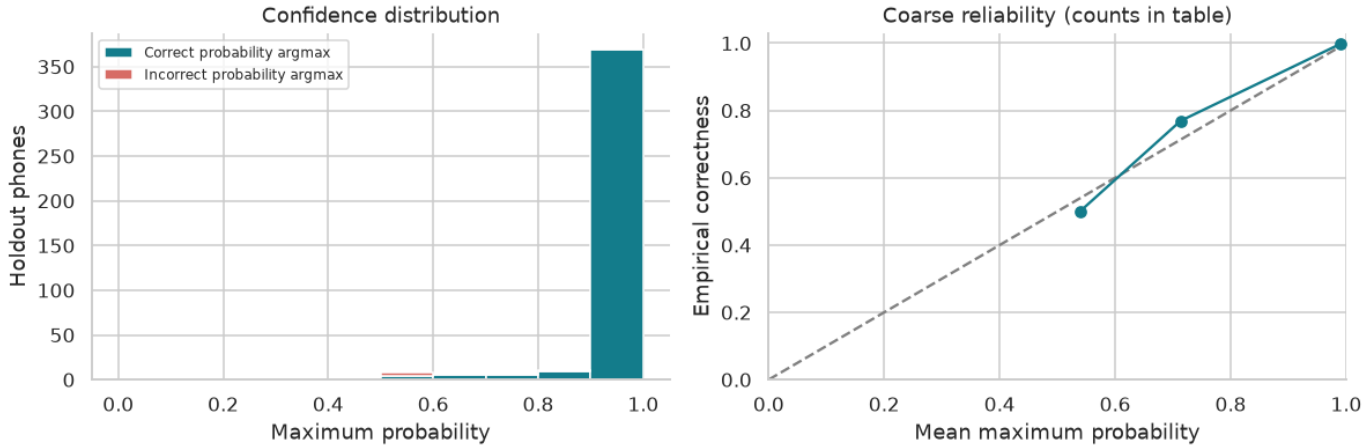
Class probabilities. `p_0`, `p_1`, `p_2` and `p_3` are the probabilities assigned to Low, Medium, High and Very high. They sum to one for each phone. The selected Logistic Regression model predicts the class with the largest probability.

We also calculate log loss and the multiclass Brier score. Log loss penalizes low probability assigned to the true class. Here Brier score is the mean sum of squared differences between the four probabilities and the one-hot target. Lower values are better. The reliability plot compares average maximum probability with the observed fraction correct in each bin.

	Log loss	Brier score	Mean max probability	Decision disagreements
0	0.0462	0.0271	0.9716	0

	Mean probability	Observed accuracy	count
bin			
(0.4, 0.6]	0.5391	0.5000	8
(0.6, 0.8]	0.7134	0.7692	13
(0.8, 1.0]	0.9896	0.9974	379

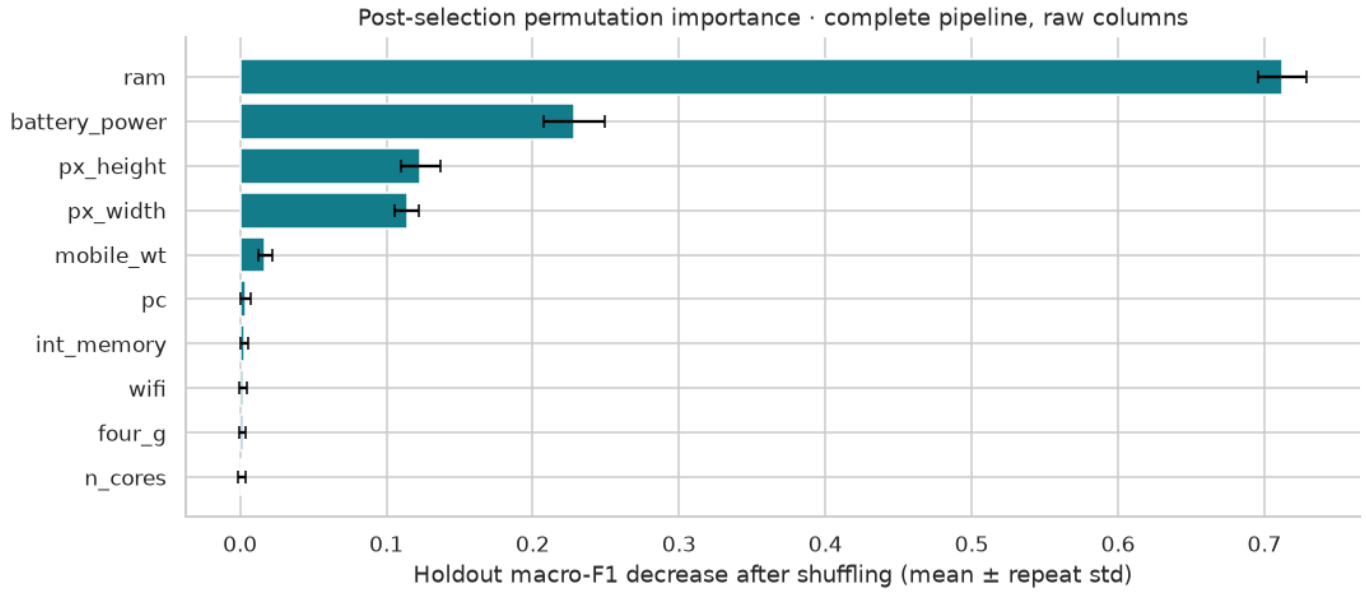
Figure 8



Reliability-bin counts help interpret the probability plot: a bin with few records has a less stable observed accuracy. The following permutation analysis shuffles one original feature at a time and measures the decrease in holdout macro-F1. It explains the selected model after evaluation; it is not used to choose new features.

	Predictor	F1 decrease	Shuffle std
0	ram	0.7124	0.0167
1	battery_power	0.2283	0.0207
2	px_height	0.1234	0.0131
3	px_width	0.1139	0.0081
4	mobile_wt	0.0172	0.0051
5	pc	0.0037	0.0032
6	int_memory	0.0028	0.0026
7	wifi	0.0020	0.0024
8	four_g	0.0018	0.0022
9	n_cores	0.0013	0.0023
10	three_g	0.0000	0.0019
11	dual_sim	-0.0002	0.0018
12	touch_screen	-0.0010	0.0012
13	sc_w	-0.0020	0.0015
14	clock_speed	-0.0022	0.0034
15	m_dep	-0.0027	0.0021
16	fc	-0.0027	0.0017
17	sc_h	-0.0030	0.0024
18	blue	-0.0032	0.0016
19	talk_time	-0.0047	0.0021

Figure 9



Top predictors: `ram`, `battery_power`, `px_height`. Shuffling passes raw columns through the full pipeline, including any derived products. Correlated variables can share importance. Negative or small drops can reflect sampling variation. Error bars show variation across repeated shuffles. A larger drop indicates greater reliance on that feature.

```
Holdout evaluation complete.
```

7. Streamlit Application

Application: Mobile Price Prediction

After evaluation, we refit the selected configuration on all **2,000 labeled records**. The reported holdout metrics belong to the development-trained model. The final fitted pipeline combines preprocessing and classification, so the app uses the same feature order and encoding as training.

The app groups the 20 inputs into performance, display, camera, connectivity, and hardware/battery settings. It returns the predicted category and four class probabilities. Batch upload accepts the supplied test data, preserves IDs and row order, validates predictors and provides downloadable results. The complete app and preprocessing source are included in the notebook cells.

	id	price_range
0	1	3
1	2	3
2	3	2
3	4	3
4	5	1
5	6	3
6	7	3
7	8	1
8	9	3
9	10	0

	predicted_test_count
price_range	
0	253
1	232
2	256
3	259

```
The final pipeline generated 1,000 predictions in the original row order.
```

```
Checks: 15 passed, 9 subtests passed in 2.29s
```

The application checks above cover prediction, probability totals, missing or invalid inputs, saved-model reloads, batch row order and the two interactive forms.

Use of the app: enter a phone's specifications and select **Predict price category**. The **What-if simulator** compares the current profile with changed RAM and battery values. **Batch CSV** accepts multiple phones; `price_range` is the predicted class and `p_0` to `p_3` give the probabilities of the four categories.

8. Additional Analysis

The what-if comparison runs the fitted model twice: once for the original profile and once after changing RAM and battery capacity. The other 18 predictors are held fixed. This makes it easy to inspect how the predicted category and probabilities change for a selected phone.

Below we use the first supplied test record, reduce RAM by 1,000 MB and increase battery capacity by 300 mAh, keeping values within the training ranges.

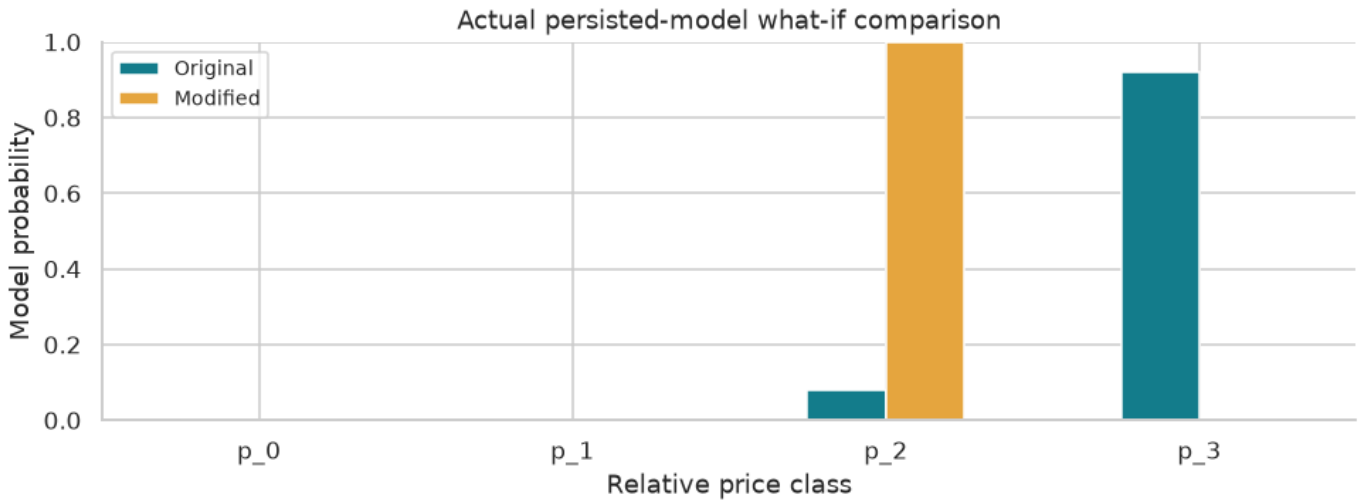
Columns 1-6 of 8

	scenario	RAM_MB	battery_mAh	price_range	p_0	p_1
0	Original test row 1	3476	1043	3	0.0	0.0000
1	Modified RAM / battery	2476	1343	2	0.0	0.0002

Columns 7-8 of 8

	p_2	p_3
0	0.0786	0.9214
1	0.9998	0.0000

Figure 10



	Feature	Implementation
0	What-if comparison	Changes RAM/battery and holds other predictors fixed
1	Batch prediction	Scores uploaded rows and preserves their IDs and order
2	Feature experiment	Compares original inputs with pixel and screen areas
3	Probability analysis	Shows class probabilities, log loss and Brier score

9. Summary and Conclusions

The following table brings together the dataset preparation, model choice and evaluation results.

	Summary	Result
0	Dataset	2,000 labeled phones; 1,000 unlabeled test phones; 20 predictors
1	Preparation	Remove empty test columns and exclude id; binary and one-hot encoding
2	Evaluation split	1,600 development / 400 holdout rows; stratified, seed 42
3	Selected classifier	Logistic Regression
4	Development CV	Macro-F1 0.9612 +/- 0.0127
5	Holdout	Accuracy 0.9800; macro-F1 0.9800
6	Holdout errors	8 errors, all 8 between adjacent classes
7	Top predictors	ram, battery_power, px_height
8	Feature engineering	Original inputs retained after the area-feature comparison
9	Balancing	No score change; the supplied classes are already balanced
10	Final model	Selected pipeline refitted on all 2,000 labeled rows
11	Application	Single prediction, batch upload, class probabilities and what-if comparison

Logistic Regression gave the highest development CV macro-F1 among the tested configurations. The voting ensemble ranked second, followed by SVM, Random Forest and Decision Tree. This suggests that a linear decision boundary captures much of the class separation in this dataset. More flexible models did not improve the measured CV score with the settings tested.

The selected model correctly classified **392 of 400 holdout records**. All eight errors were between neighboring price classes. Class 2 had the lowest F1 (0.9700), while every class had recall of at least 0.97. Training macro-F1 was about 0.985, compared with 0.961 in CV and 0.980 on holdout. These results do not show a large training-to-holdout gap; the holdout is also a smaller sample than the CV evaluation.

RAM was the strongest predictor in the exploratory ranking and permutation analysis, followed by battery capacity and pixel height. The area-feature experiment did not meet our retention threshold, and balancing methods produced the same scores because the labels were already balanced. We therefore kept the original 20 predictors and the natural class distribution.

The final application uses the selected pipeline fitted on all labeled rows and supports single-phone prediction, batch prediction and a RAM/battery comparison.